

Isolation Level	Dirty Reads	Lost Update	Nonrepeatable Reads	Phantom Reads
Read Uncommitted	Yes	Yes	Yes	Yes
Read Committed	No	Yes	Yes	Yes
Repeatable Read	No	No	No	Yes
Snapshot	No	No	No	No
Serializable	No	No	No	No

Read Uncommitted

Example:

Transaction-1

set transaction Isolation level read uncommitted

begin tran

update person set name='Raj4' where Id=2

rollback

Transaction-2

set transaction Isolation level read uncommitted

select * from person

Read Committed

Close all the windows

Transaction-1

--Alter database [IsolationDB] Set Read_committed_snapshot on

set transaction isolation level read committed --default

Begin tran

update person set name='test' where id=2

commit

Set transaction isolation level Read Committed --default

Transaction-2

set transaction isolation level read committed --default

select * from person where id=2

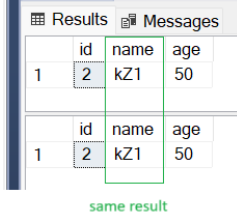
Repeatable Read

Non-Repeatable Read Issue Transaction-1 Begin tran select * from person where id=2 waitfor delay '00:00:05' select * from person where id=2 commit	Transaction-2 update person set name='kZ1' where id=2
Result:  Different Result	(1 row affected) Completion time: 2025-04-19T19:21:27.9044490+05:30

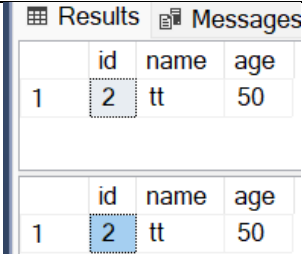
Fixed (Transaction-1 running while Transaction-2 waiting for completion of Transaction-1)

Transaction-1 lock for id=2 only, we can also lock for range of records i.e. id>2

Transaction-1 set transaction isolation level repeatable read Begin tran select * from person where id=2 waitfor delay '00:00:05' select * from person where id=2 commit	Transaction-2 update person set name='YZ1' where id=2
---	--

 Same Result	(1 row affected) Completion time: 2025-04-19T19:26:18.0128875+05:30
--	---

Repeatable Read (lost data issue fixed)

set transaction isolation level repeatable read Begin tran select * from person where id=2 waitfor delay '00:00:05' update person set name='tt' where id=2 select * from person where id=2 commit	set transaction isolation level repeatable read Begin tran select * from person where id=2 waitfor delay '00:00:03' update person set name='zt' where id=2 select * from person where id=2 commit
	(1 row affected) Msg 1205, Level 13, State 51, Line 7 Transaction (Process ID 70) was deadlocked on lock resources with another process and has been chosen as the deadlock victim. Rerun the transaction. Completion time: 2025-04-19T19:38:13.6854130+05:30

But phantom read issue

set transaction isolation level repeatable read Begin tran select * from person where id between 2 and 7 waitfor delay '00:00:05' update person set name='tt' where id=2 select * from person where id between 2 and 7 commit	set transaction isolation level repeatable read Begin tran insert into person(name,age) values ('Phantom2',66) commit
--	---

Results

Messages

	id	name	age
les	2	tt	50
2	3	Y	50
3	4	Y	15
4	6	Phantom	66

	id	name	age
1	2	tt	50
2	3	Y	50
3	4	Y	15
4	6	Phantom	66
5	7	Phantom2	66

(1 row affected)

Completion time: 2025-04-19T19:44:48.1435487+05:30

Serializable

It will fix the phantom read issue

<pre>set transaction isolation level serializable Begin tran select * from person where id between 2 and 10 waitfor delay '00:00:05' update person set name='xy' where id=2 select * from person where id between 2 and 10 commit</pre>	<pre>set transaction isolation level serializable Begin tran insert into person(name,age) values ('Phantom3',66) commit</pre>																																
ResultsMessages <table><thead><tr><th></th><th>id</th><th>name</th><th>age</th></tr></thead><tbody><tr><td>1</td><td>2</td><td>tt</td><td>50</td></tr><tr><td>on_temp</td><td></td><td>Y</td><td>50</td></tr><tr><td>3</td><td>4</td><td>Y</td><td>15</td></tr></tbody></table> <table><thead><tr><th></th><th>id</th><th>name</th><th>age</th></tr></thead><tbody><tr><td>1</td><td>2</td><td>xy</td><td>50</td></tr><tr><td>2</td><td>3</td><td>Y</td><td>50</td></tr><tr><td>3</td><td>4</td><td>Y</td><td>15</td></tr></tbody></table> Same no of records no phantom read		id	name	age	1	2	tt	50	on_temp		Y	50	3	4	Y	15		id	name	age	1	2	xy	50	2	3	Y	50	3	4	Y	15	(1 row affected) Completion time: 2025-04-19T19:54:30.8604360+05:30
	id	name	age																														
1	2	tt	50																														
on_temp		Y	50																														
3	4	Y	15																														
	id	name	age																														
1	2	xy	50																														
2	3	Y	50																														
3	4	Y	15																														

-----End of ACID-----

-----Some Example-----



<https://www.youtube.com/watch?v=jD0c4X0tSc8>

Default is Read Committed.

Repeatable Read (lost data issue fixed)

A screenshot of the SQL Server Enterprise interface showing two query windows and their message panes. The left window, 'SQLQuery6.sql', contains a transaction with a delay and an update. The right window, 'SQLQuery2.sql', contains a transaction that reads the data and then updates it. The message pane for the right window shows a deadlock error.

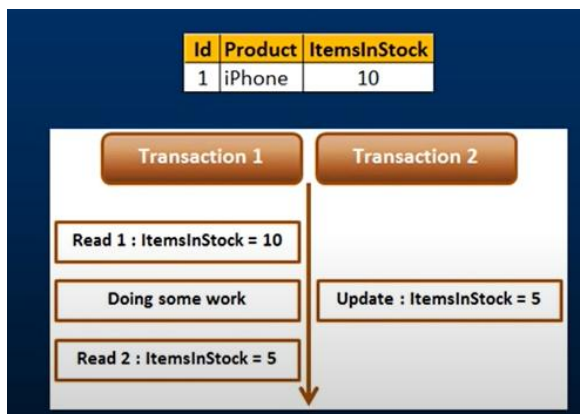
```
SQLQuery6.sql - (...KAT-PC\Tan (54))
--Select * from tblInventory
Set Transaction Isolation level Repeatable Read
-- Transaction 1
Begin Transaction
Declare @ItemsInStock int
Select @ItemsInStock = ItemsInStock
from tblInventory where Id=1
Waitfor Delay '00:00:10'
Set @ItemsInStock = @ItemsInStock - 1
Update tblInventory
Set ItemsInStock = @ItemsInStock where Id=1
Print @ItemsInStock
Commit Transaction

Messages
(1 row(s) affected)
9
```

```
SQLQuery2.sql - (...KAT-PC\Tan (52))
-- Transaction 2
Set Transaction Isolation level Repeatable Read
Begin Transaction
Declare @ItemsInStock int
Select @ItemsInStock = ItemsInStock
from tblInventory where Id=1
Waitfor Delay '00:00:1'
Set @ItemsInStock = @ItemsInStock - 2
Update tblInventory
Set ItemsInStock = @ItemsInStock where Id=1
Print @ItemsInStock
Commit Transaction

Messages
Msg 1206, Level 13, State 51, Line 12
Transaction (Process ID 52) was deadlocked on lock resources with another process and has
```

Non-repeatable issue



```
SQLQuery1.sql - (...KAT-PC\Tan (53))
-- Transaction 1
Begin Transaction
Select ItemsInStock from tblInventory
where Id = 1
waitfor delay '00:00:10'
Select ItemsInStock from tblInventory
where Id = 1
commit transaction
```

Results

ItemsInStock
10

ItemsInStock
5

```
SQLQuery1.sql - (...KAT-PC\Tan (54))
-- transaction 2
Update tblInventory set ItemsInStock = 5
where Id = 1
```

Messages

(1 row(s) affected)

Non-repeatable issue fixed

The screenshot displays two SQL queries being executed in SQL Server Enterprise Manager. The left query window, titled 'SQLQuery1.sql - (...KAT-PC\Tan (53))', contains the following code:

```
--Select * from tblInventory  
-- Transaction 1  
set transaction isolation level repeatable read  
Begin Transaction  
Select ItemsInStock from tblInventory  
where Id = 1  
  
waitfor delay '00:00:10'  
  
Select ItemsInStock from tblInventory  
where Id = 1
```

The right query window, titled 'SQLQuery1.sql - (...KAT-PC\Tan (54))', contains the following code:

```
-- transaction 2  
Update tblInventory set ItemsInStock = 5  
where Id = 1  
  
wait until complete the transaction 1
```

The Results pane on the left shows the output of the first query, which is a single row with the value 10. The Messages pane on the right shows the output of the second query, which is '(1 row(s) affected)'.

ItemsInStock
10

ItemsInStock
10

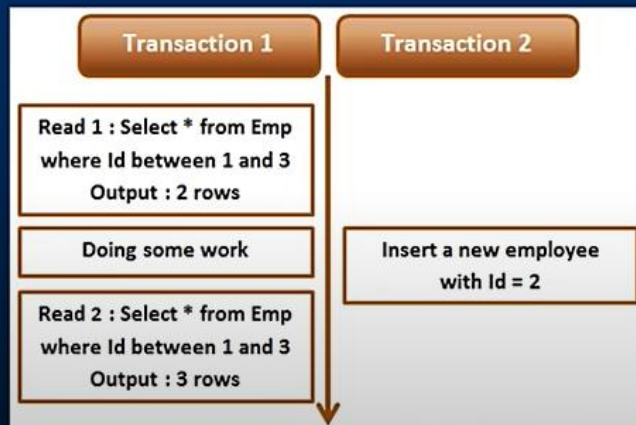
Phantom reads example in SQL Server

If you are in need of the DVD with all the videos and PPT's, please visit
<http://pragimtech.com/order.aspx>

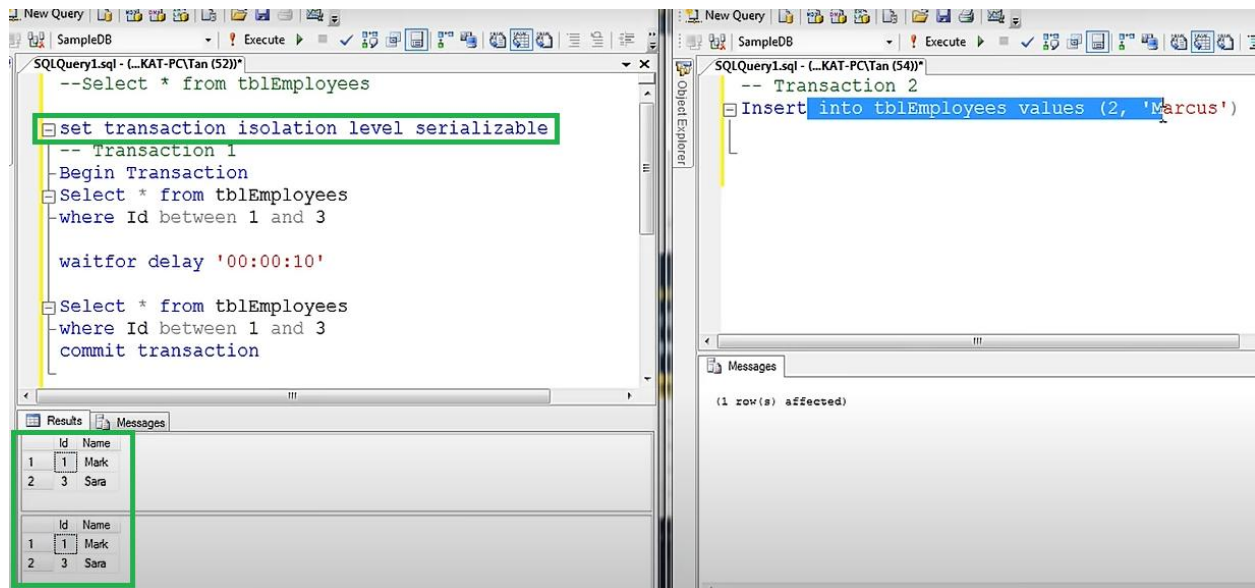
Phantom read happens when one transaction executes a query twice and it gets a different number of rows in the result set each time. This happens when a second transaction inserts a new row that matches the WHERE clause of the query executed by the first transaction.

Employees Table

Id	Name
1	Mark
3	Sara
100	Mary



Serialization



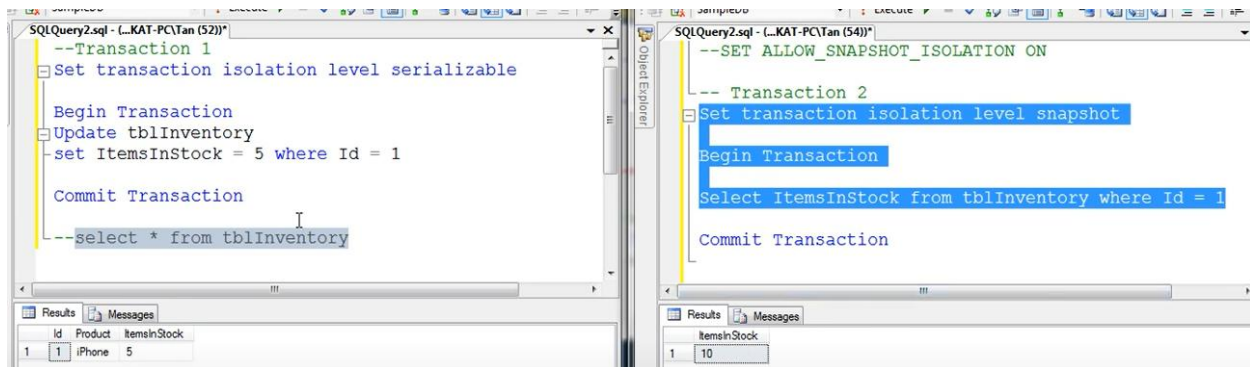
Range lock the above screenshot

The below does not lock but need to use

Transaction 2:

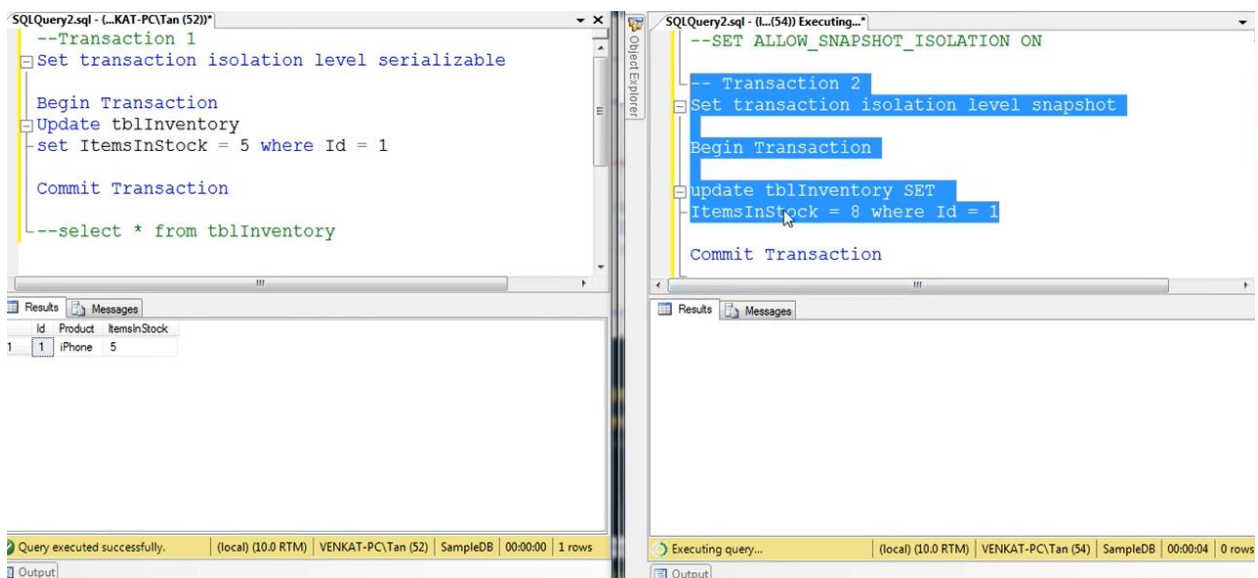
```
SQLQuery2.sql - (...KAT-PC\Tan (54))*
--Alter database SampleDB
--SET ALLOW_SNAPSHOT_ISOLATION ON

-- Transaction 2
Set transaction isolation level snapshot
```

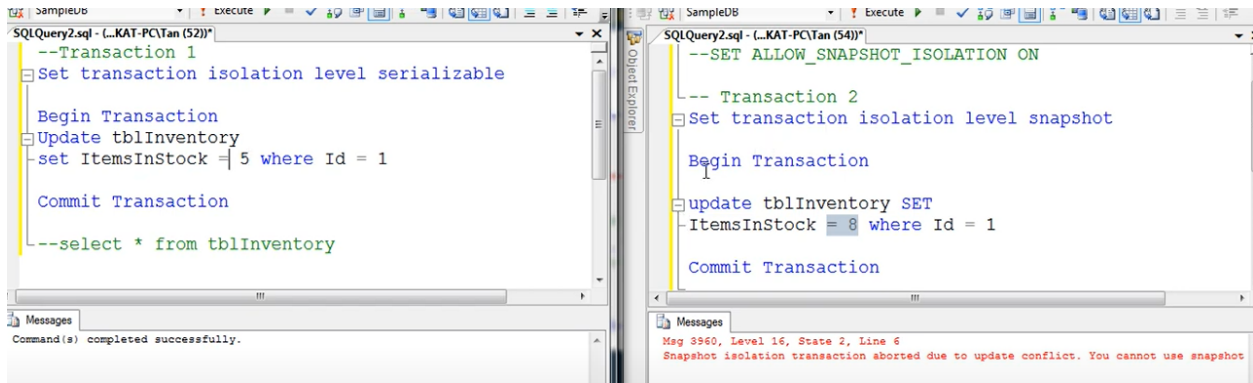


The above transaction does not lock but using the versioning.

The below is it is locked (TR. 2) the data by updating by transaction 1



It is preventing the updated lost.



Read committed snapshot isolation level is not a different isolation level. It is a different way of implementing Read committed isolation level. One problem we have with Read Committed isolation level is that, it blocks the transaction if it is trying to read the data, that another transaction is updating at the same time.

To use READ_COMMITTED_SNAPSHOT isolation, enable it at the database level

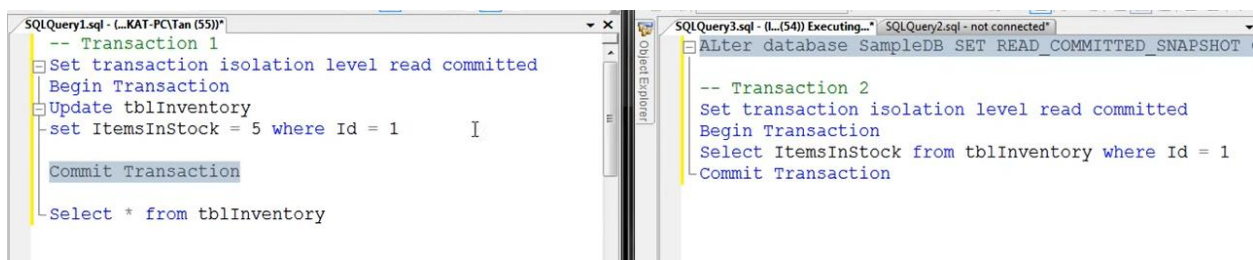
Alter database SampleDB SET READ_COMMITTED_SNAPSHOT ON

```
--Transaction 1
Set transaction isolation level Read Committed
Begin Transaction
Update tblInventory set ItemsInStock = 5 where Id = 1
waitfor delay '00:00:10'
Commit Transaction

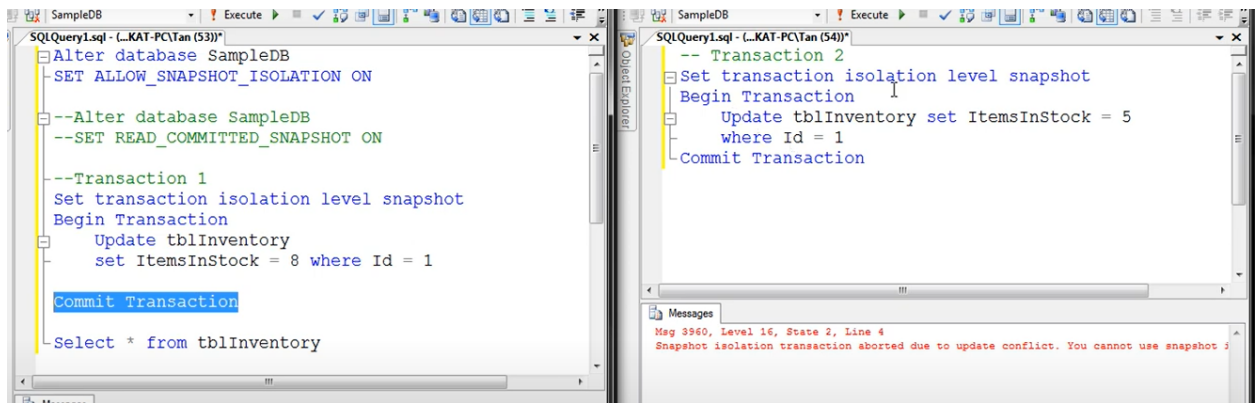
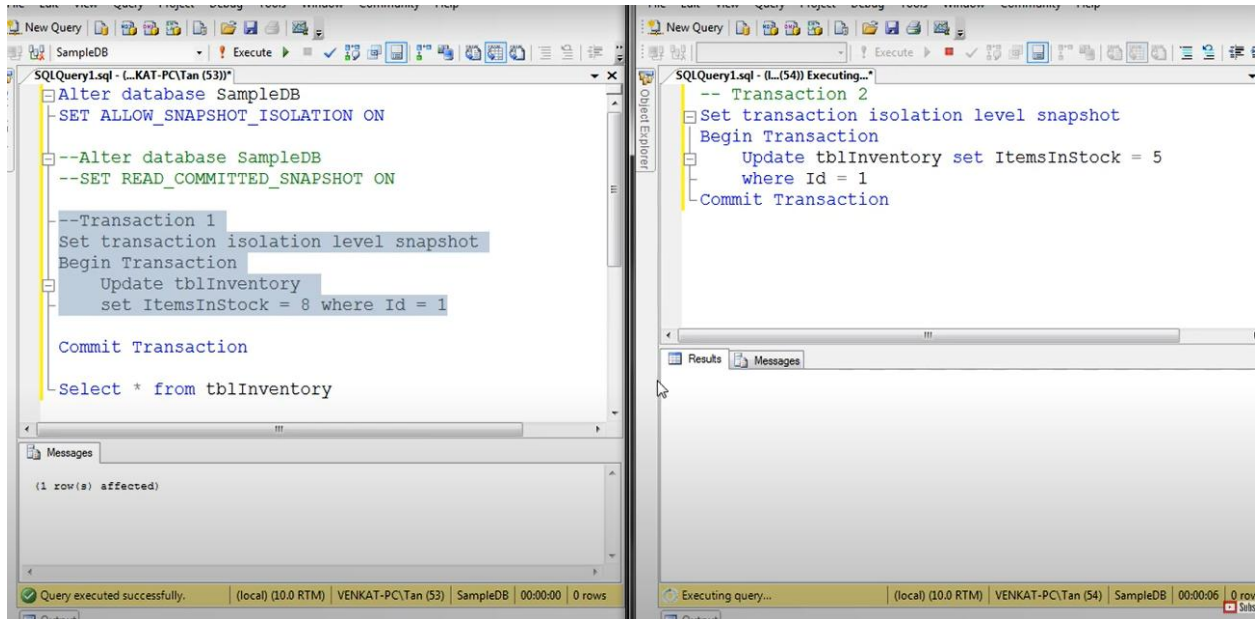
-- Transaction 2
Set transaction isolation level read committed
Begin Transaction
Select ItemsInStock from tblInventory where Id = 1
Commit Transaction
```

Id	Name	ItemsInStock
1	iPhone	10

Alter database Set Read_committed_snapshot on



Transaction 2 is blocked



No update conflict

Read_Committed_snapshot on

The screenshot displays the SQL Server Enterprise Manager interface. The central Query Editor window is split into two panes. The left pane contains SQL code for Transaction 1, and the right pane contains SQL code for Transaction 2. Below the code, the Messages window shows the execution results for Transaction 2.

Transaction 1 Code:

```
Alter database SampleDB
SET ALLOW_SNAPSHOT_ISOLATION OFF

Alter database SampleDB
SET READ_COMMITTED_SNAPSHOT ON

--Transaction 1
Set transaction isolation level read committed
Begin Transaction
Update tblInventory
    set ItemsInStock = 8 where Id = 1
Commit Transaction

Select * from tblInventory
```

Transaction 2 Code:

```
-- Transaction 2
Set transaction isolation level read committed
Begin Transaction
Update tblInventory set ItemsInStock = 5
    where Id = 1
Commit Transaction
```

Messages Window:

(1 row(s) affected)